FIITMeteo IoT Protocol: PoC Documentation

1. Introduction

This document describes the proof-of-concept (PoC) of a custom application protocol over UDP for FIITMeteo battery-powered weather sensors, implemented with readable JSON messages for the first submission.

The solution consists of two Python programs:

- **server.py**: the central server. It registers sensors, issues session tokens, checks data integrity, acknowledges and displays measurements, and monitors sensor activity with KEEPALIVE messages.
- **tester.py**: one program that emulates any number of sensors of the four types (ThermoNode, WindSense, RainDetect, AirQuality). It registers them, sends measurements every 10 s, retries unacknowledged messages and can inject errors or send user-entered values.

The PoC implements all requirements P1–P15 and the acceptance tests UAT 1–5. The message format is JSON so the protocol can be debugged in Wireshark and in the logs; the final submission will replace it with a compact binary encoding. Both programs use only the Python standard library and the `socket` module directly (P5).

Sensor type	Parameters (range, precision)
ThermoNode	temperature −50.0–60.0 °C (0.1), humidity 0.0–100.0 % (0.1), dew_point −50.0–60.0 °C (0.1), pressure 800.00–1100.00 hPa (0.01)
WindSense	wind_speed 0.0–50.0 m/s (0.1), wind_gust 0.0–70.0 m/s (0.1), wind_direction 0–359° (integer), turbulence 0.0–1.0 (0.1)
RainDetect	rainfall 0.0–500.0 mm (0.1), soil_moisture 0.0–100.0 % (0.1), flood_risk 0–3 (integer), rain_duration 0–60 min (integer)
AirQuality	co2 300–5000 ppm (integer), ozone 0.0–500.0 µg/m³ (0.1), air_quality_index 0–500 (integer)

2. System architecture

The server listens on one UDP socket; the tester uses one UDP socket for all emulated sensors and tells them apart by sequence number, sensor ID and token.

Addresses. The server binds to an IP and port chosen in its menu (default `0.0.0.0:5500`, i.e. all interfaces). The tester binds to `0.0.0.0` with port 0, so the operating system picks a free port; the server replies to the source address of each received datagram (`recvfrom`) and stores it in the session for KEEPALIVE messages.

Sessions and tokens (P3, P10). A sensor is identified by the pair (sensor type, sensor ID), which must be unique. After a successful registration the server creates a session and returns a random 12-character hexadecimal token (`secrets.token_hex(6)`). Every DATA, KEEPALIVE and KEEPALIVE answer carries the token; a message with an unknown token is rejected. When a session is cancelled, the token becomes invalid and a new registration returns a new token.

Sequence numbers. Each program has its own counter. The tester uses one counter for all sensors, so every message it sends has a unique number; replies reference it in the `ack-for` field. The server numbers its own messages (ACK, REJECT, RETRANSMIT, KEEPALIVE) with a separate counter.

Threads. Both programs keep the menu responsive by running the network work in background threads. Shared data (sessions, pending messages, counters) is protected by a `threading.Lock`.

Program	Thread	Job
server	main	menu: configuration, start/stop, display data, ACK suppression
server	receive (`run`)	receives datagrams and handles REGISTER, DATA and ACK
server	keepalive (`keepaliveLoop`)	every 0.1 s: SENSOR IDLE, KEEPALIVE every 5 s, SENSOR DISCONNECTED
tester	main	menu: configuration, registration, toggles, custom data, error injection, session end
tester	receive (`listening`)	receives replies and KEEPALIVE messages and dispatches them
tester	retry (`retransmitLoop`)	every 0.1 s: resends messages without ACK after 3 s, drops them after 3 attempts
tester	session timer (`sessionTimerLoop`)	ends a sensor's session after 30 s without any message from the server
tester	one per sensor (`sendLoop`)	sends that sensor's DATA every 10 s while sending is ON

3. Protocol messages (JSON)

The protocol has six message types; each is one UDP datagram with one JSON object encoded in UTF-8.

Every message carries `type` and `sequence`. Replies carry `ack-for` with the `sequence` of the message they answer. A received datagram that is not valid UTF-8 JSON or not a JSON object is logged and ignored.

Field	Type	Used in	Meaning
type	string	all	REGISTER, ACK, REJECT, DATA, RETRANSMIT or KEEPALIVE
sequence	integer	all	sender's message number (separate counter in the server and the tester)
ack-for	integer	ACK, REJECT, RETRANSMIT	sequence of the message being answered
sensorType	string	REGISTER	ThermoNode, WindSense, RainDetect or AirQuality
sensorId	string	REGISTER, DATA, KEEPALIVE, ACK to KEEPALIVE	sensor identifier, unique within its type, 1–16 characters

Field	Type	Used in	Meaning
token	string	ACK to REGISTER, DATA, KEEPALIVE, ACK to KEEPALIVE	session token, 12 hexadecimal characters
reason	string	REJECT	why the message was refused
crc	integer	DATA	CRC-32 of the message without the `crc` field
sensor values	number	DATA	one field per parameter, named in lowercase as in the specification

3.1 REGISTER (tester → server)

The sensor announces its type and ID and asks for a session.

```
{"type": "REGISTER", "sequence": 1, "sensorType": "ThermoNode",
"sensorId": "T-01"}
```

3.2 ACK (server → tester)

Confirms a message. The answer to REGISTER also carries the new token; the answer to DATA confirms that the measurement was accepted.

```
{"type": "ACK", "ack-for": 1, "token": "ccc1742a0657", "sequence": 1}
{"type": "ACK", "ack-for": 5, "sequence": 5}
```

3.3 REJECT (server → tester)

Refuses a message. Reasons: `Invalid sensor` (unknown type or empty ID), `Already registered` (the type and ID already have a session), `Invalid token`, `token doesn't match sensor Id`, `Missing values` (a parameter of the sensor type is absent).

```
{"type": "REJECT", "ack-for": 2, "reason": "Invalid token", "sequence":
4}
```

3.4 DATA (tester → server)

Carries one measurement of one sensor, the session token and the CRC-32 integrity value. Example for each sensor type:

```
{"type": "DATA", "sequence": 5, "sensorId": "T-01", "token":
"ccc1742a0657",
 "temperature": 21.3, "humidity": 55.4, "dew_point": 11.9, "pressure":
1013.25,
 "crc": 1476104302}
{"type": "DATA", "sequence": 6, "sensorId": "W-01", "token":
"bf765bab1079",
 "wind_speed": 9.8, "wind_gust": 24.5, "wind_direction": 314,
"turbulence": 0.3,
 "crc": 2087211546}
{"type": "DATA", "sequence": 7, "sensorId": "R-01", "token":
"d786f157ba04",
 "rainfall": 172.1, "soil_moisture": 56.0, "flood_risk": 2,
```

```
"rain_duration": 29,
 "crc": 3391870528}
{"type": "DATA", "sequence": 8, "sensorId": "A-01", "token":
"7fc8a48f42da",
 "co2": 4711, "ozone": 19.4, "air_quality_index": 456, "crc": 912843176}
```

3.5 RETRANSMIT (server → tester)

The DATA message referenced by `ack-for` failed the integrity check; the sensor sends the same message (same sequence, original values) again.

```
{"type": "RETRANSMIT", "ack-for": 9, "sequence": 9}
```

3.6 KEEPALIVE (server → tester) and its answer

The server checks whether an idle sensor is still reachable. The sensor answers with an ACK that references the KEEPALIVE's sequence and carries its ID and token.

```
{"type": "KEEPALIVE", "sequence": 12, "sensorId": "T-01", "token":
"ccc1742a0657"}
{"type": "ACK", "sequence": 12, "ack-for": 12, "sensorId": "T-01",
"token": "ccc1742a0657"}
```

3.7 CRC-32 calculation

The CRC is computed in the same way on both sides, so equal content gives an equal value:

1. Take the message without the `crc` field.
2. Serialize it canonically: `json.dumps(body, sort_keys=True, separators=(",", ":"))`, with keys sorted and no spaces.
3. Encode it as UTF-8 and compute `zlib.crc32` (the CRC-32 polynomial 0x04C11DB7, as in the Ethernet FCS).

The CRC values in the DATA examples above are illustrative except for T-01, which was computed this way.

4. Protocol behaviour

Every sensor follows the same life cycle: register, send data every 10 s, and keep the session alive until the sensor or the server ends it.

4.1 Registration (P3, P10)

1. The tester sends REGISTER with the sensor type and ID.
2. The server checks that the type is one of the four known types and the ID is not empty; otherwise it answers REJECT `Invalid sensor`.
3. If the pair (type, ID) already has a session, it answers REJECT `Already registered`.
4. Otherwise it creates a session, logs the registration and answers ACK with the token.
5. On ACK the tester stores the token, starts the 30 s session timer and starts automatic sending (P4). On REJECT it removes the sensor.

REGISTER is retried like DATA (section 4.4).

4.2 Data transfer (P4, P8)

Each sensor has its own thread that sends a DATA message every 10 s with random values within the allowed ranges and precision; the first one 10 s after registration. Automatic sending can be switched off and on per sensor; the thread keeps running, so the sensor still answers KEEPALIVE.

The user can also send one extra DATA message with values typed in the menu (P8). Every value is checked for type, number of decimal places and range before it is sent; automatic sending of all sensors continues.

The server checks each DATA message in this order and stops at the first failure:

1. CRC integrity → RETRANSMIT (section 4.3)
2. token belongs to an active session → otherwise REJECT `Invalid token`
3. `sensorId` matches the token's session → otherwise REJECT
4. all parameters of the sensor type are present → otherwise REJECT `Missing values`
5. accept: store the values, update the time of the last data, answer ACK

A DATA message whose sequence the server has already stored (a resend after a lost ACK) is acknowledged again but not stored twice.

4.3 Integrity check and retransmission (P6, P7, UAT 3)

The tester adds a CRC-32 to every DATA message (section 3.7). The server recomputes it; on a mismatch it logs `Integrity check failed for <type>:<id> retransmission requested`, does not store the values and answers RETRANSMIT. The tester resends the same message from its pending list, with the same sequence and the original values.

Error injection (P7): a menu option marks one sensor. Its next DATA message is copied and one value is changed after the CRC was computed, so the check fails on the server. No separate message announces the error, and the stored original stays intact for the retransmission.

4.4 ACK timeout and retries (P11, UAT 5)

Every message that expects an answer (REGISTER, DATA) is kept in the tester's pending list with the number of attempts and the time it was last sent. A thread checks the list every 0.1 s:

Time after first send	No ACK yet → action
0 s	message sent (attempt 1)
3 s	same message sent again (attempt 2)
6 s	same message sent again (attempt 3)
9 s	message dropped and logged

An ACK, REJECT or RETRANSMIT with the matching `ack-for` removes the message from the list.

For UAT 5 the server menu can suppress the ACK for the next x DATA messages (1–5) of a chosen sensor. Every received attempt, repeats included, uses up one; afterwards the server answers normally. With x = 5, message 1 is sent 3 times without an answer, message 2 twice, and its third attempt is acknowledged.

4.5 Keepalive and end of session (P9, UAT 4)

The server checks every session every 0.1 s, measured from the time of the sensor's last DATA message:

Time since last data	Server action
15 s	logs SENSOR IDLE, sends KEEPALIVE 1
20 s	sends KEEPALIVE 2
25 s	sends KEEPALIVE 3
30 s	3 unanswered: logs SENSOR DISCONNECTED, deletes the session, the token becomes invalid

- An answer (ACK with the KEEPALIVE's sequence and the token) resets the missed counter; the session stays, and KEEPALIVE continues every 5 s.
- New DATA ends the IDLE state (`SENSOR ACTIVE again`) and stops KEEPALIVE.
- The tester answers KEEPALIVE for every active session, also when automatic sending is OFF.
- Ending a sensor's session in the tester menu deletes its token and stops its data and its KEEPALIVE answers; the server disconnects it exactly 30 s after its last data. A new registration then gets a new token.

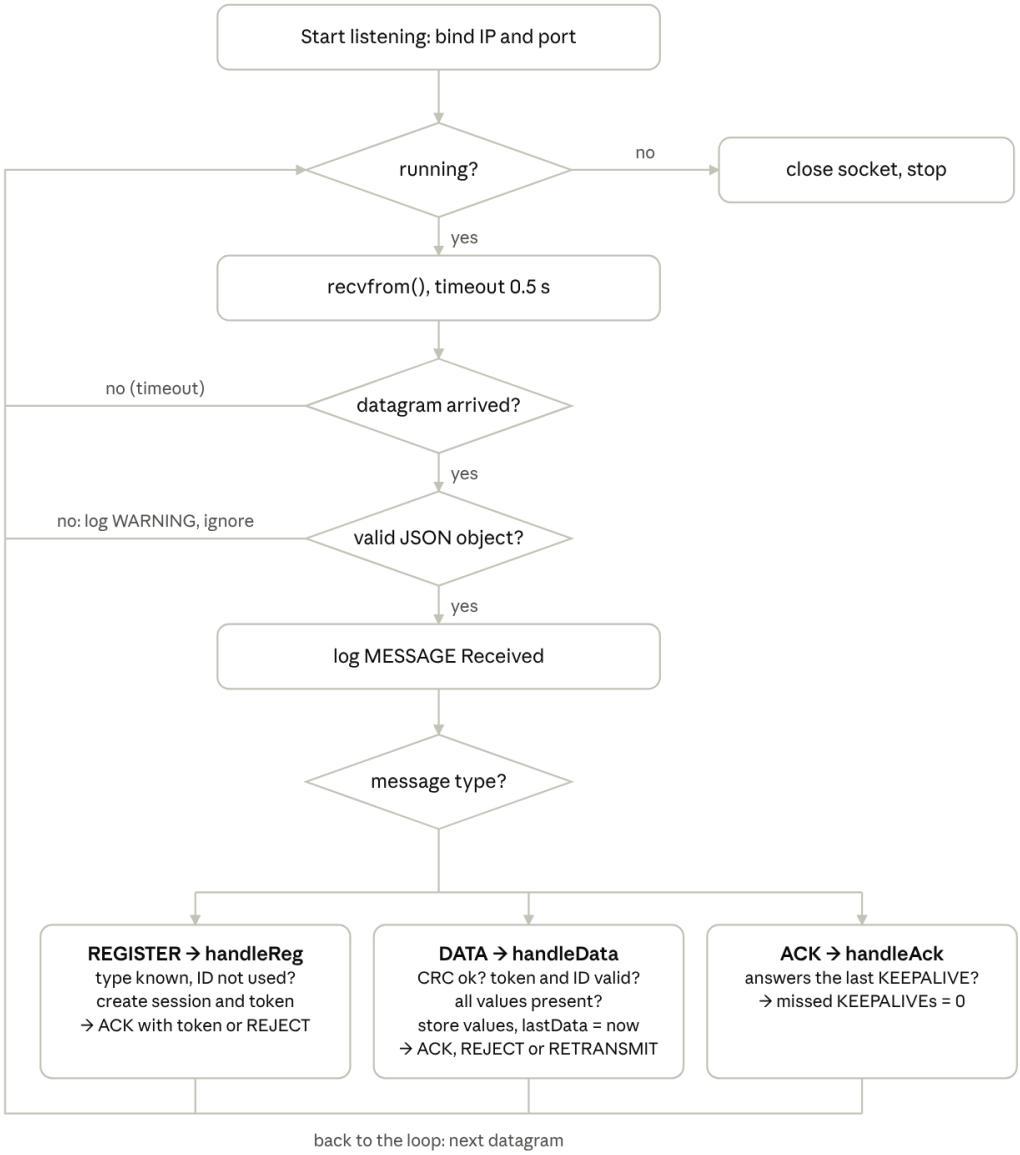
Sensor timer. Each sensor ends its own session if it receives nothing from the server for 30 s. The assignment counts this from the last KEEPALIVE; because the server sends no KEEPALIVE while data flows, every reply from the server (ACK, RETRANSMIT, KEEPALIVE) restarts the timer, otherwise a working sensor would end its session every 30 s. The timer also covers a disconnected cable. If the server rejects DATA with `Invalid token`, the tester ends that session as well, so the sensor can register again.

5. Flowcharts

Each program has a receive loop and one or more send loops, each running in its own thread beside the menu.

5.1 Server receive loop

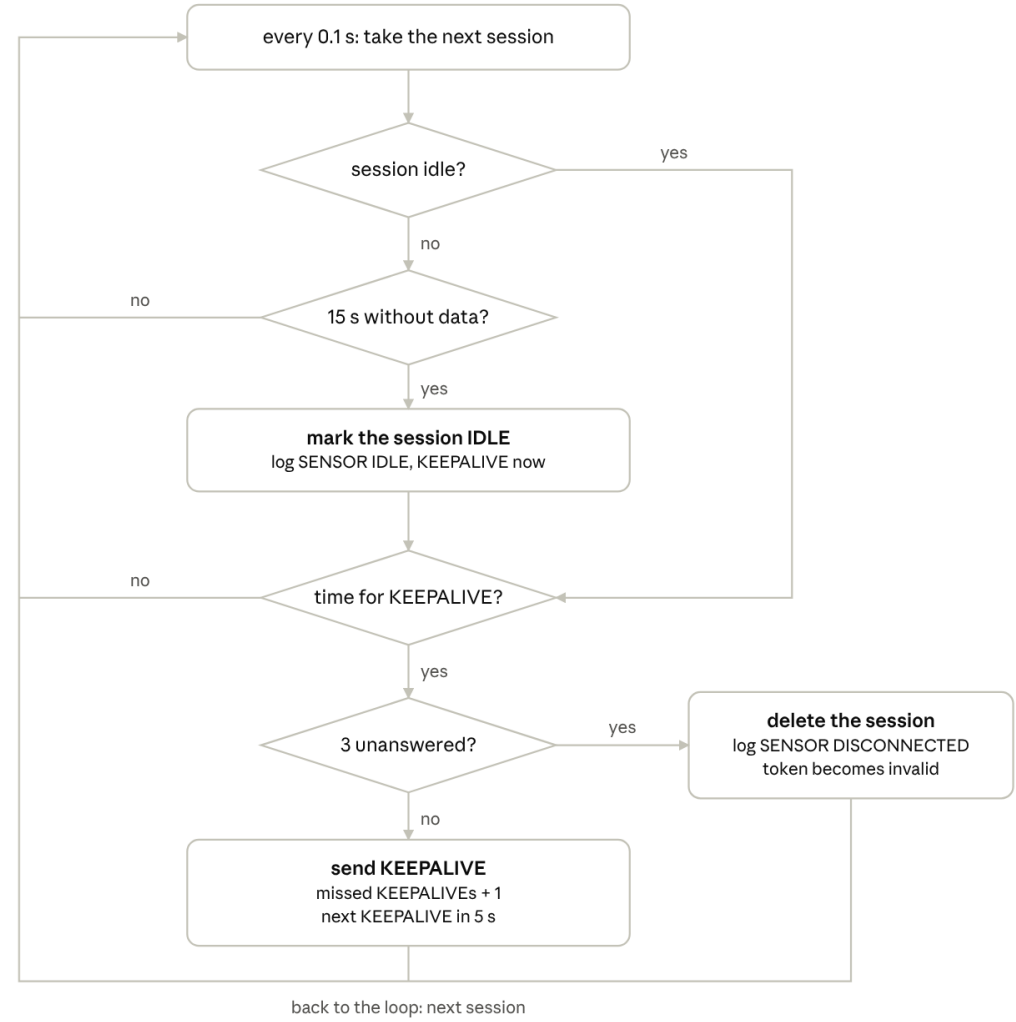
Server receive loop: every datagram is checked, then handled by its type



The receive thread waits for a datagram with a 0.5 s timeout, so it notices a stop request within half a second. A datagram that is not a valid JSON object is logged and ignored; every other one is logged and handed to the handler for its type. All replies (ACK, REJECT, RETRANSMIT) are sent from these handlers to the sender’s address.

5.2 Server send loop (keepalive)

Server keepalive loop: 15 s idle, KEEPALIVE every 5 s, disconnect at 30 s

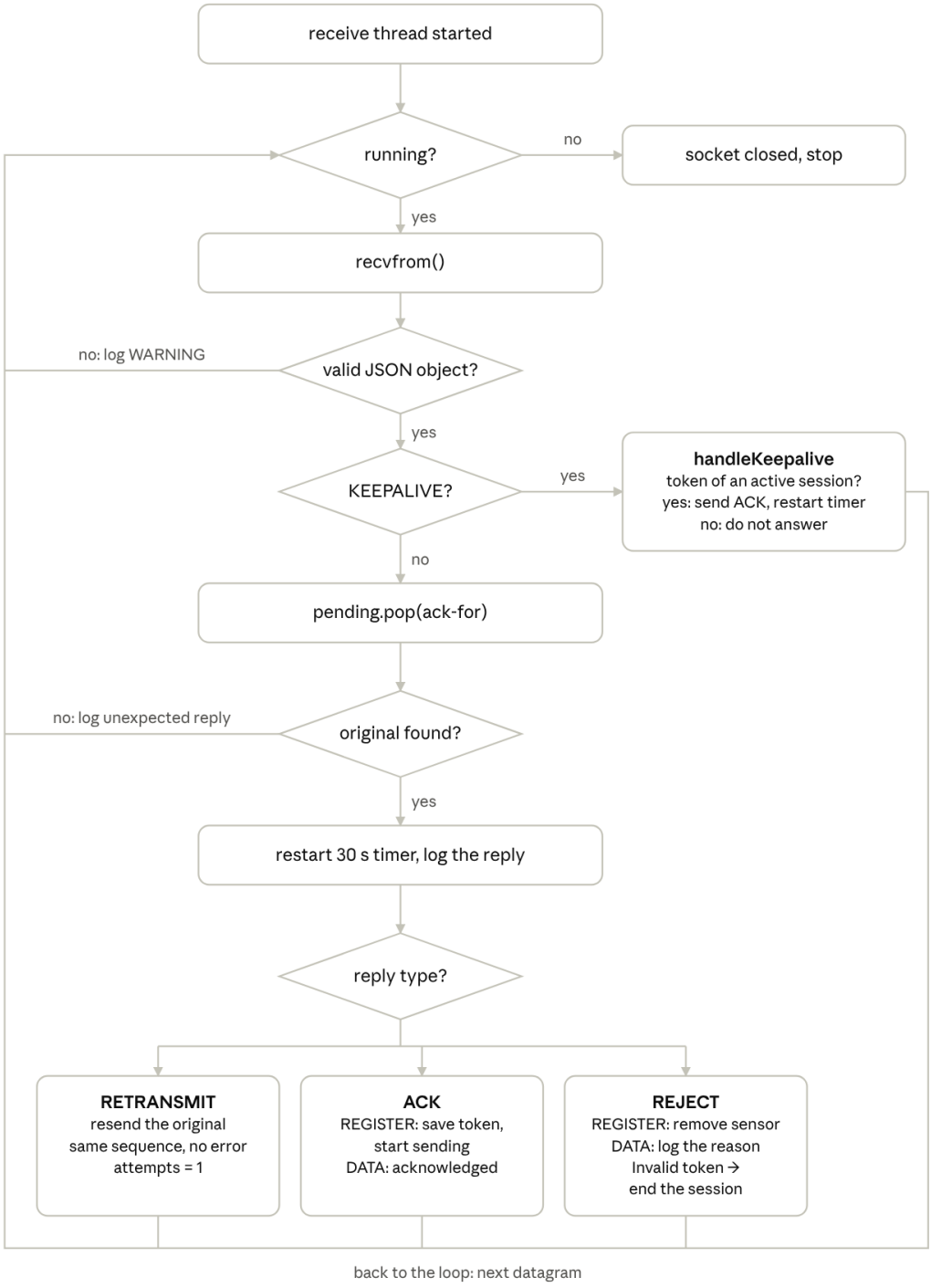


ACK, REJECT and RETRANSMIT are sent by the receive thread as replies to received messages.

The only messages the server starts on its own are KEEPALIVE messages. Every 0.1 s the keepalive thread checks all sessions: after 15 s without data a session becomes IDLE and gets a KEEPALIVE every 5 s; an answer resets the missed counter, and the third unanswered KEEPALIVE plus 5 s ends the session, 30 s after the last data.

5.3 Tester receive loop

Tester receive loop: KEEPALIVE is answered, replies are matched by ack-for

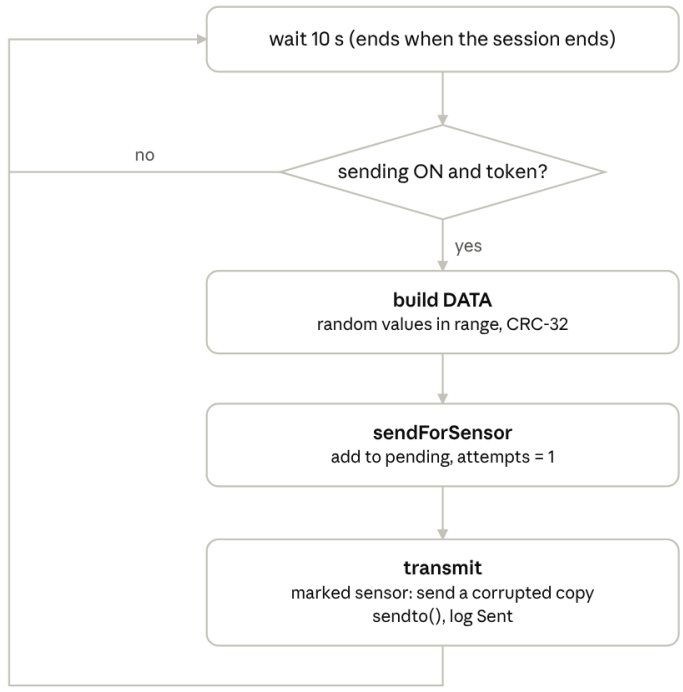


One receive thread serves all emulated sensors. A KEEPALIVE is matched to a sensor by its token and answered only while that sensor's session is active. Every other message is a reply: its `ack-for` finds the original message in the pending list, which tells the tester which sensor and which message it answers. Every reply restarts the sensor's 30 s session timer.

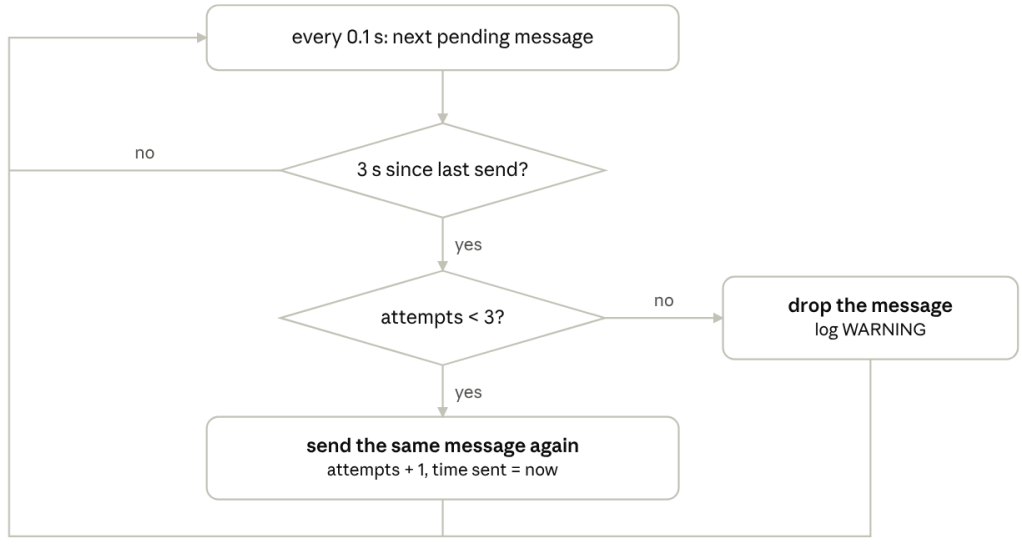
5.4 Tester send loops

Tester send loops: DATA every 10 s, a repeat after 3 s without an ACK

a) Automatic sending, one thread per sensor



b) Retries, one thread for all pending messages



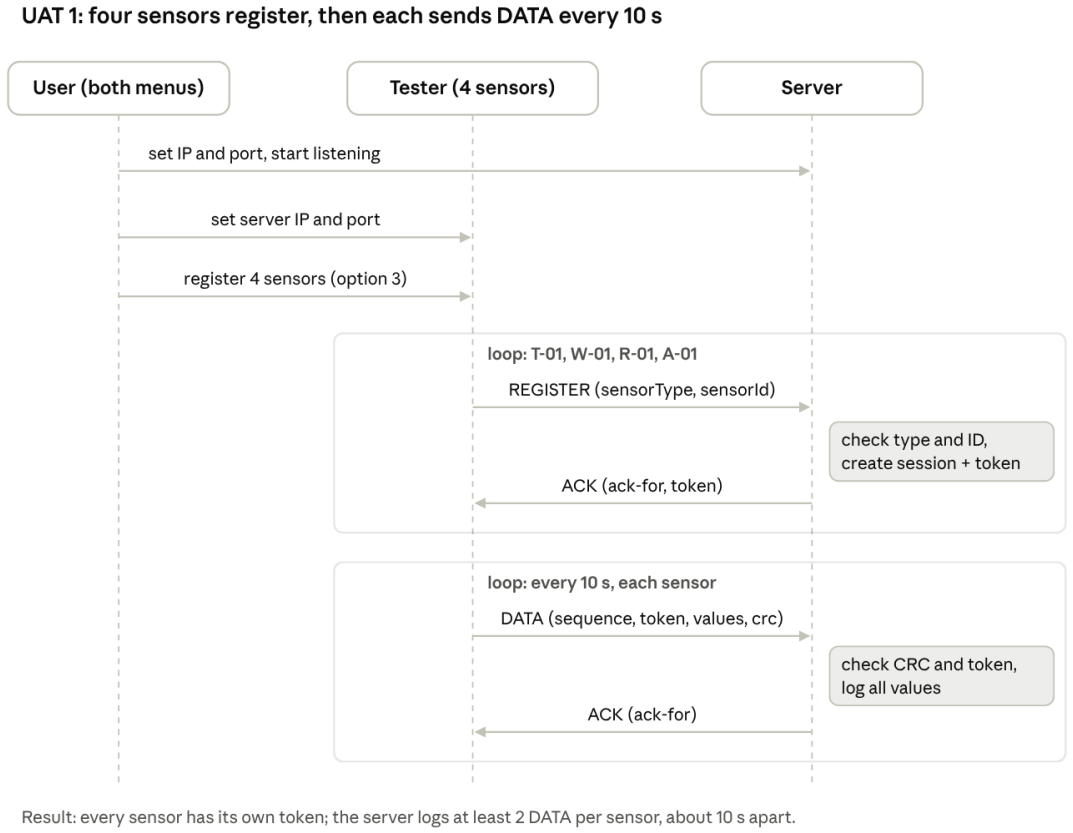
A reply with a matching ack-for removes the message from pending; the 30 s session timer runs in its own thread.

Each registered sensor has a thread that sends DATA every 10 s while sending is ON; custom data from the menu goes through the same `sendForSensor` and `transmit` steps. Every message that expects an answer waits in the pending list; the retry thread repeats it 3 s after the last send and drops it 3 s after the third send. A third thread (`sessionTimerLoop`, every 0.5 s) ends a sensor's session when nothing has come from the server for 30 s.

6. Sequence diagrams

The diagrams show the messages of UAT 1 and UAT 2 at the level of the protocol; each registration and data exchange is drawn once inside a loop.

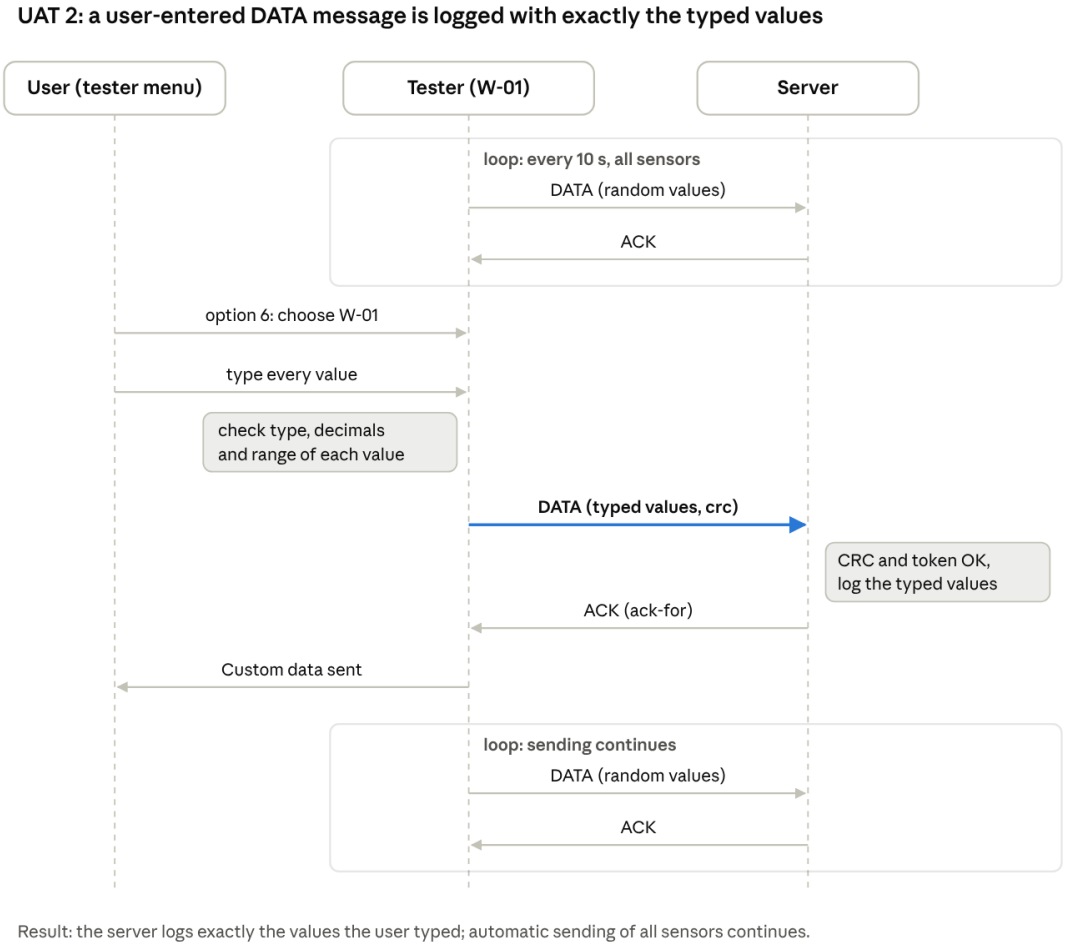
6.1 UAT 1: registration and automatic data



Result: every sensor has its own token; the server logs at least 2 DATA per sensor, about 10 s apart.

The user starts the server, sets the server address in the tester and registers one sensor of each type. Each REGISTER is answered with an ACK carrying the sensor’s token, after which the sensor sends DATA every 10 s and the server answers each with an ACK.

6.2 UAT 2: custom data message



Result: the server logs exactly the values the user typed; automatic sending of all sensors continues.

While all sensors send automatically, the user chooses a sensor and types every parameter. The tester checks each value before sending one extra DATA message; the server checks it like any other DATA, logs exactly the typed values and acknowledges it.

7. Logging, user interface and error handling

7.1 Logging (P12, P15)

Both programs always write their logs to a file (`server.log`, `tester.log`); a menu option also prints them to the console. Every line has the required format, with the time in UTC and milliseconds:

```
<TIMESTAMP> | <TYPE> | <LEVEL> | <SOURCE> | <EVENT/MESSAGE>
2026-10-21T21:12:15.912Z | SYSTEM | INFO | SERVER | SENSOR IDLE:
ThermoNode:T-01
2026-10-21T21:12:15.913Z | MESSAGE | INFO | TESTER:ThermoNode:T-01 |
Sent:
    {"type":"ACK","sequence":12,"ack-
for":12,"sensorId":"T-01","token":"ccc1742a0657"}
```

- **TYPE**: `SYSTEM` for local events (start, errors, SENSOR IDLE, SENSOR DISCONNECTED, retries), `MESSAGE` for every sent and received protocol message, logged in full with all sensor values.

- **LEVEL**: syslog severities (RFC 5424): `INFO`, `WARNING` (integrity failure, retry, drop, suppressed ACK, disconnect), `ERROR` (failed send, failed start).
- **SOURCE**: `SERVER`, or `TESTER:<type>:<id>` for events of one sensor (`TESTER` for events of the whole program).

The logger is shared by all threads; a lock makes each line be written whole.

7.2 Server menu

Option	Action
1	set the listening IP and port (validated; `0.0.0.0` = all interfaces)
2	start listening
3	show registered sensors: address, active/IDLE, seconds since last data, latest values (P2)
4	console logging on/off
5	stop listening (the port can be reused immediately)
6	do not ACK the next x DATA messages of a chosen sensor (UAT 5)
0	exit

7.3 Tester menu

Option	Action
1	set the server IP and port (validated)
2	register one sensor: choose the type, enter the ID
3	register one sensor of each type (T-01, W-01, R-01, A-01)
4	show sensors: token or state, sending on/off
5	automatic sending on/off for a chosen sensor (P4)
6	send custom data for a chosen sensor (P8, UAT 2)
7	console logging on/off
8	corrupt the next DATA message of a chosen sensor (P7, UAT 3)
9	end the local session of a chosen sensor (UAT 4)
0	exit

Typing `q` in any prompt cancels the action and returns to the menu.

7.4 Error handling (P13, P14)

Situation	Handling
invalid IP address	checked with `ipaddress.ip_address`; asked again
port outside 1024–65535 or not a number	asked again
port already in use, or IP not on this computer	`bind` error caught, message shown, program stays in the menu
datagram that is not valid JSON	logged as WARNING and ignored
unexpected error while handling one message	logged as ERROR, the receive thread continues
failed send (network down)	logged as ERROR; the retry mechanism resends
invalid menu choice or value out of range	message shown, asked again
server unreachable	message retried 3 times then dropped; session ends after 30 s without contact

8. Libraries used

Both programs use only the Python standard library; no ready-made communication or protocol library is used (P5).

Library	server	tester	Used for
`socket`	yes	yes	creating and binding the UDP socket, `sendto`, `recvfrom`, receive timeout
`json`	yes	yes	serializing and parsing messages; canonical form for the CRC
`threading`	yes	yes	receive, keepalive, retry, timer and send threads; `Lock` for shared data; `Event` to stop a sensor's thread
`ipaddress`	yes	yes	validating IP addresses entered in the menu
`datetime`	yes	yes	UTC timestamps with milliseconds in the logs
`time`	yes	yes	`monotonic()` clock for timeouts, IDLE detection and retries
`zlib`	yes	yes	`crc32` integrity check
`secrets`	yes	no	random session tokens (`token_hex`)
`collections` (`deque`)	yes	no	the last 50 sequence numbers per sensor, to recognize repeated DATA
`random`	no	yes	random sensor values within the allowed ranges

9. Proposal for measuring efficiency (JSON vs. binary)

The JSON and binary versions will be compared on the same test scenario by the number of bytes each puts on the wire, including the Ethernet, IPv4 and UDP layers; a ThermoNode DATA message currently has an efficiency of about 2.7 %.

9.1 What is counted

Each datagram travels in one Ethernet frame, so the bytes on the link are:

Layer	Size	Note
Ethernet II header	14 B	destination MAC, source MAC, EtherType
IPv4 header	20 B	no options
UDP header	8 B	ports, length, checksum
application payload	variable	our JSON (now) or binary (final) message
Ethernet FCS	4 B	CRC-32 trailer; usually not shown by Wireshark
preamble + SFD, inter-frame gap	8 B + 12 B	physical layer; reported separately as “on the wire”

Useful data = only the sensor values, counted at the smallest size their range and precision allow. For example ThermoNode: temperature has 1101 possible values (11 bits), humidity 1001 (10 bits), dew_point 11 bits, pressure 30 001 (15 bits), together 47 bits, i.e. 6 B. Likewise WindSense 32 bits (4 B), RainDetect 31 bits (4 B), AirQuality 35 bits (5 B).

frame = 14 + 20 + 8 + payload + 4 B

η = useful data bytes / frame bytes × 100 %

overhead share = (frame − useful data) / frame

9.2 Measurement procedure

1. Run the server and the tester on two computers; start a Wireshark capture with the filter `udp.port == 5500`.
2. Run the same fixed scenario for both versions: register the 4 sensor types, let each send 10 automatic DATA messages, then one KEEPALIVE cycle (sending OFF for 20 s). No errors injected, so both captures contain the same messages.
3. Export the packet list (File → Export Packet Dissections → CSV) with the columns `frame.len`, `udp.length` and the message type, and add 4 B FCS to every frame.
4. Calculate per message type: payload size, frame size, efficiency η and overhead share; and for the whole scenario: total bytes and total useful bytes.
5. Extrapolate one sensor’s daily traffic: 8640 DATA + ACK exchanges per day (one every 10 s).
6. Compare the two versions in one table and state the saving in % of bytes per day; fewer bytes means less radio time and longer battery life.

9.3 Baseline: JSON message sizes

Computed from the current message format (Python `json.dumps` with default separators, 12-character token):

Message	Payload	Frame incl. FCS	Useful data	η
DATA ThermoNode	174 B	220 B	6 B	2.7 %
DATA WindSense	175 B	221 B	4 B	1.8 %
DATA RainDetect	175 B	221 B	4 B	1.8 %
DATA AirQuality	148 B	194 B	5 B	2.6 %
REGISTER	83 B	129 B	none	none
ACK to REGISTER	69 B	115 B	none	none
ACK to DATA	44 B	90 B	none	none
RETRANSMIT	51 B	97 B	none	none
KEEPALIVE	82 B	128 B	none	none
ACK to KEEPALIVE	91 B	137 B	none	none

One ThermoNode DATA + ACK exchange uses 310 B for 6 B of measurements (η = 1.9 %), which is about 2.7 MB per sensor per day. The 46 B of lower-layer headers and FCS are fixed, so the binary version can only shrink the payload: field names, quotes, decimal text and the 12-character token are where the savings are. The real sizes will be confirmed in the Wireshark captures.